

Process scheduling

Operating Systems – EDA093/DIT401

Vincenzo Gulisano
vincenzo.gulisano@chalmers.se



CHALMERS
UNIVERSITY OF TECHNOLOGY



UNIVERSITY OF
GOTHENBURG

What to read (main textbook)

- Chapter 2.4, 8.1.1, 8.1.2, 8.1.4, 10.3.4, 11.4.1

(optional additional reading: 5.1-5.7, 1.10 from Silberschatz
Operating System Concepts)

Objectives

- Explore in greater depth processes, threads and their scheduling / execution
- Discuss different types of systems (batch/interactive/real-time)
- Discuss challenges of multiprocessor/multicore architectures

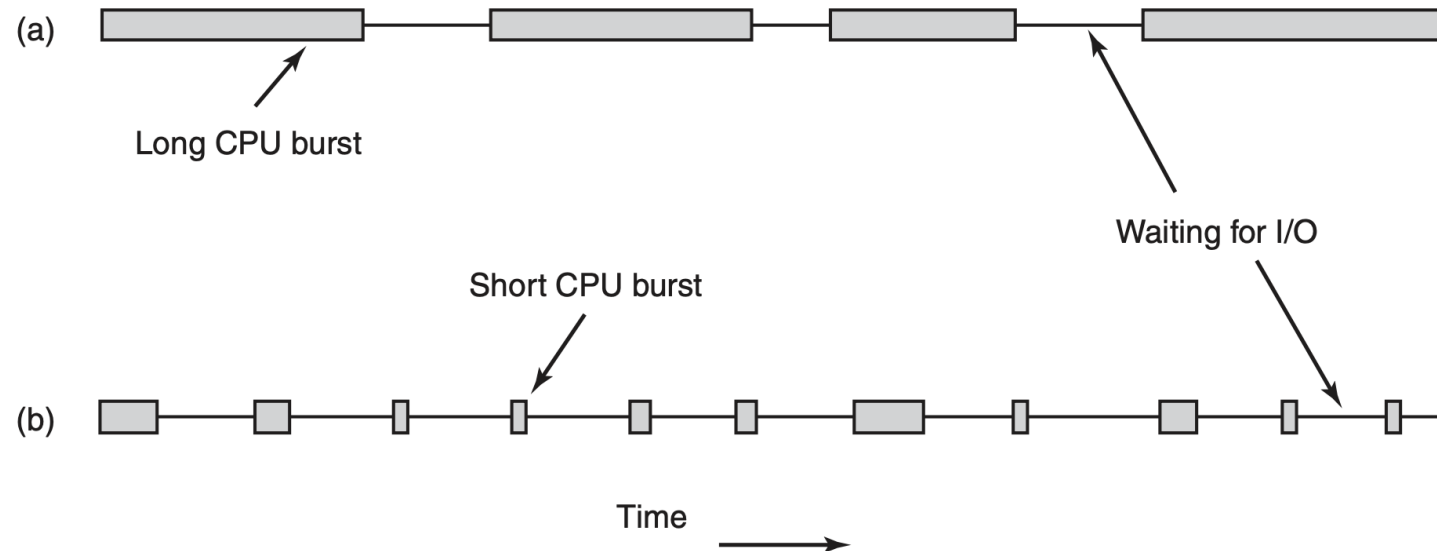
Agenda

- Introduction
- Scheduling in batch, interactive, and real-time systems
- Process vs. thread scheduling
- Multiprocessor hardware
 - Why does it complicate the matter?
- Multiprocessor scheduling
 - Time sharing
 - Space sharing
 - Gang scheduling

Agenda

- Introduction
- Scheduling in batch, interactive, and real-time systems
- Process vs. thread scheduling
- Multiprocessor hardware
 - Why does it complicate the matter?
- Multiprocessor scheduling
 - Time sharing
 - Space sharing
 - Gang scheduling

Introduction



- Two types of processes:
 - a) CPU-bound (or compute-bound)
 - b) I/O-bound
- Notice: I/O-bound does not mean “I/O is bound because the I/O is slow”, it means few CPU-core cycles between I/O requests

When can the OS make scheduling decisions?

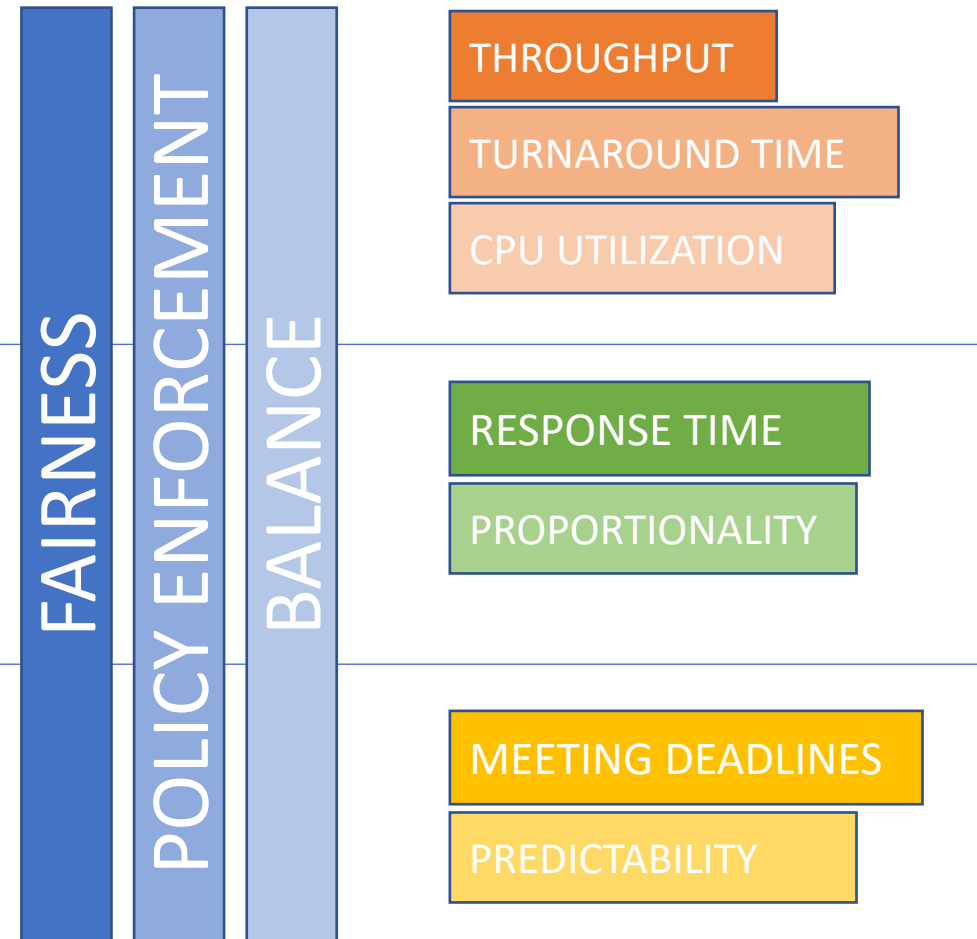
- Process termination
 - I/O request
 - Process creation
 - I/O interrupt
 - Elapsed time
- Non-preemptive
- preemptive
-
- The diagram uses blue curly braces to group the events. The first two events, 'Process termination' and 'I/O request', are grouped under the label 'Non-preemptive'. The remaining three events, 'Process creation', 'I/O interrupt', and 'Elapsed time', are grouped under the label 'preemptive'.

Agenda

- Introduction
- Scheduling in batch, interactive, and real-time systems
- Process vs. thread scheduling
- Multiprocessor hardware
 - Why does it complicate the matter?
- Multiprocessor scheduling
 - Time sharing
 - Space sharing
 - Gang scheduling

Scheduling algorithms – Categories and goals

- **Batch:** “business-world” applications, data analysis. Usually appropriate for non-preemptive (but can also be preemptive)
- **Interactive:** Many users who need responsiveness, requiring preemptive scheduling
- **Real-time:** defined by timing constraints and deadlines, and they are often periodic or sporadic long-running tasks



A large orange circle is positioned on the left side of the slide, partially cut off by the edge.

In the following, we cover some scheduling algorithms for some of these categories, see the textbook for the other algorithms

Batch systems – scheduling

Algorithm	+	-
First-come, first-served (non-preemptive)	Easy, fair	Possibly inefficient (esp. for I/O bound processes)
Shortest Job First (non-preemptive)	Optimal for turnaround (when all jobs are available together and their lengths are known)	Starvation + need to know or predict CPU-burst lengths
Shortest Remaining Time Next (preemptive)	New short jobs get good service	Starvation + need to know or predict CPU-burst lengths

Example of inefficient first-come, first-served scheduling

- Process 0 (CPU-bound): 1 I/O every 1 second of computation, 1000 seconds to finish
- Processes 1...1000 (I/O-bound): need to perform 1000 I/O operations each (actual I/O service time and device contention are ignored)
- FCFS: Processes 1...1000 get to perform 1 I/O every second. Hence, they finish after 1000 seconds (>16 minutes)
- If the scheduler preempted Process 0 every 10 milliseconds, the other processes could complete in 10 seconds...

Interactive systems – scheduling

- Round-robin 
- Priority scheduling 
- Multiple queues
- Shortest process next
- Guaranteed scheduling
- Lottery scheduling 
- Fair-share scheduling

Round-Robin

- Quantum: time interval during which the process can run
- Process still running at the end of the quantum? Preempt!
- Simple to implement (keep a list...)
- Challenge: what's the right quantum length?
 - Too short → high overhead
 - Too long → poor responsiveness (e.g., 50th process of a batch scheduled in round-robin with a 100 milliseconds quantum waits approximately 5 seconds to start. What if it was the shortest I/O-bound of the 50 processes?)

Priority scheduling

- Not all processes are equally important
- Priorities:
 - Static (by OS or user)
 - Dynamic (by OS or user)
- Priority scheduling can be combined with round-robin scheduling → priority classes

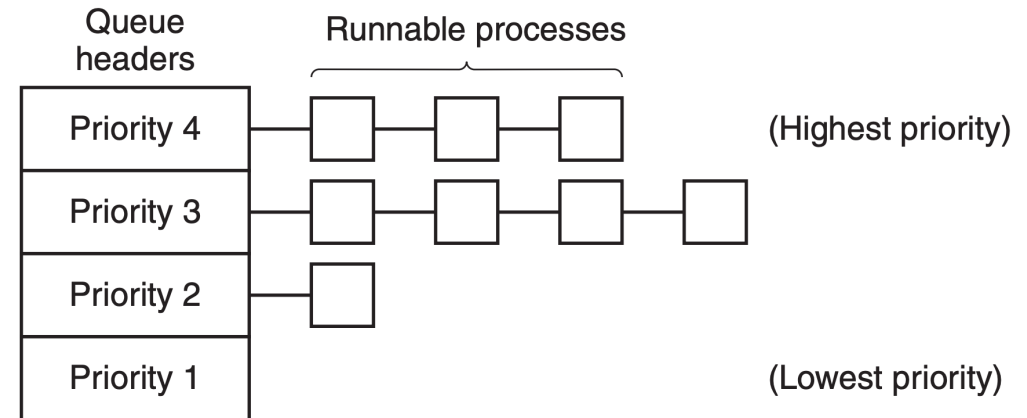


Figure 2-43. A scheduling algorithm with four priority classes.

...In the previous example (FCFS batch) I/O-bound processes could have higher priority than CPU-bound processes...

Lottery scheduling

- Alternative to priority scheduling that still gives more resources to some processes rather than others (but note that the proportional share is expected, not guaranteed):
 - Processes get “lottery tickets”.
 - Next process to run is the one holding the next randomly chosen ticket.
- Easier to map portions of resources to give to a process (i.e., portion of tickets to give) than with priority scheduling

Agenda

- Introduction
- Scheduling in batch, interactive, and real-time systems
- **Process vs. thread scheduling**
- Multiprocessor hardware
 - Why does it complicate the matter?
- Multiprocessor scheduling
 - Time sharing
 - Space sharing
 - Gang scheduling

User-level vs. Kernel-level threads

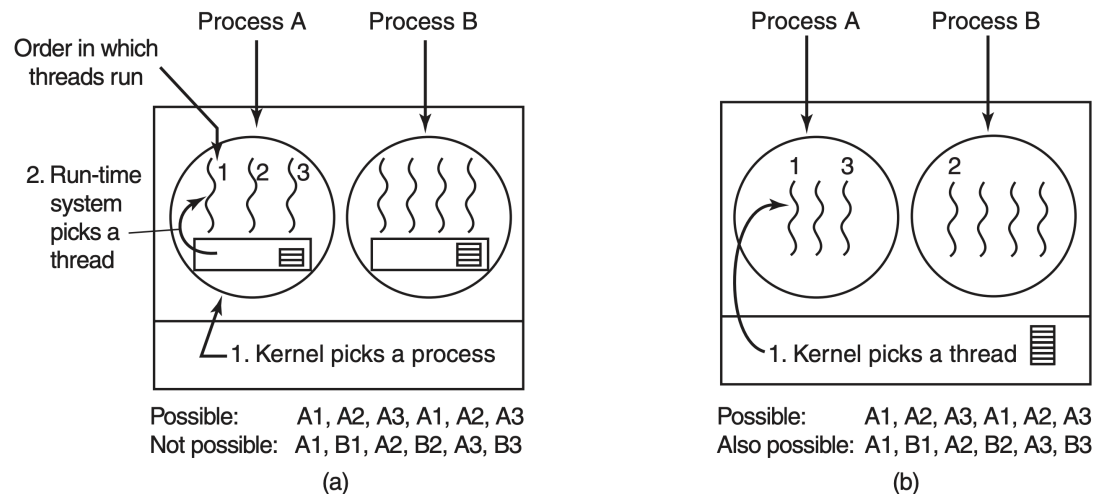


Figure 2-44. (a) Possible scheduling of user-level threads with a 50-msec process quantum and threads that run 5 msec per CPU burst. (b) Possible scheduling of kernel-level threads with the same characteristics as (a).

	User-level	Kernel-level
+	- Intra-quantum thread switch is extremely fast (context switch does not require kernel mediation) - Can employ an application-specific scheduler	Process can keep running even if one of its threads blocks for I/O
-	A thread blocking on I/O means the entire process does	Thread switch costs more (but the OS knows that switching between threads of different processes costs more than switching between threads of the same processes)

Agenda

- Introduction
- Scheduling in batch, interactive, and real-time systems
- Process vs. thread scheduling
- **Multiprocessor hardware**
 - Why does it complicate the matter?
- Multiprocessor scheduling
 - Time sharing
 - Space sharing
 - Gang scheduling

Multiprocessor hardware – Why is scheduling complicated?

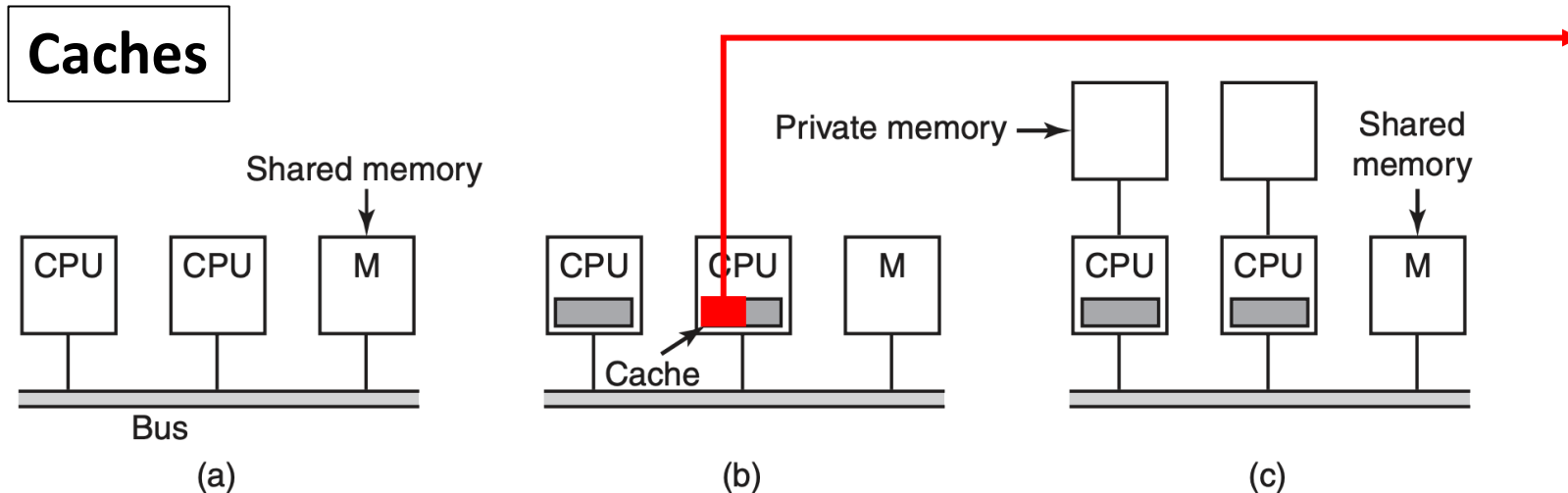


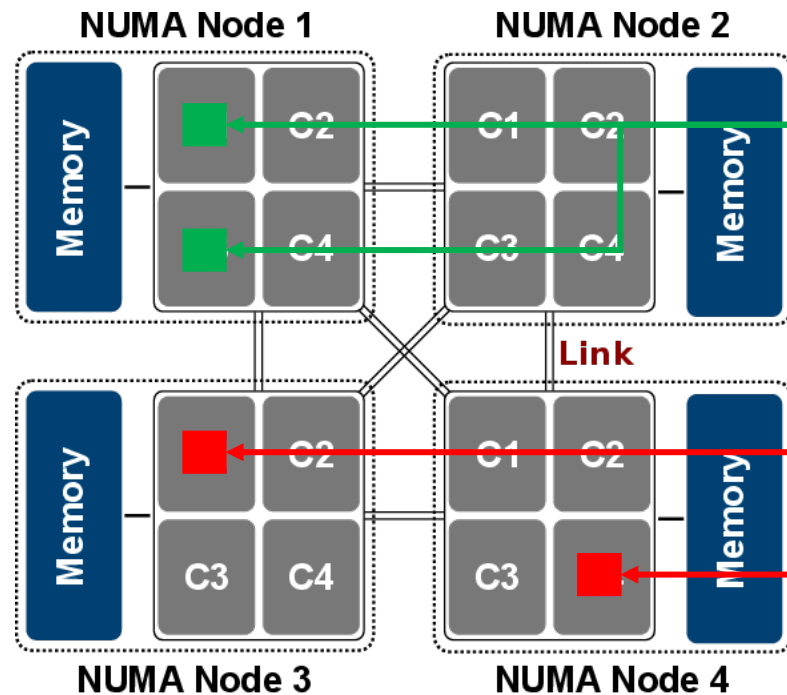
Figure 8-2. Three bus-based multiprocessors. (a) Without caching. (b) With caching. (c) With caching and private memories.

Suppose a thread has some data here and issues an I/O request. When later rescheduled, it might perform better on the CPU-core used previously (cached data can be reused) than on a new one...

... but for that we need to keep track of more such information and make it part of the scheduling process.

Multiprocessor hardware – Why is scheduling complicated?

NUMA architectures



Suppose two threads (producer/consumer) are scheduled at the same time...

Scheduling on the same socket leads to higher chances of them performing better than...

Scheduling on two different sockets

To complicate a bit further... how would the OS know that two threads are producer/consumer? (the OS might be aware of relationships, but not application semantics)

Multiprocessor hardware – Why is scheduling complicated?

Where to place the OS itself?

Each CPU refers to a dedicated OS

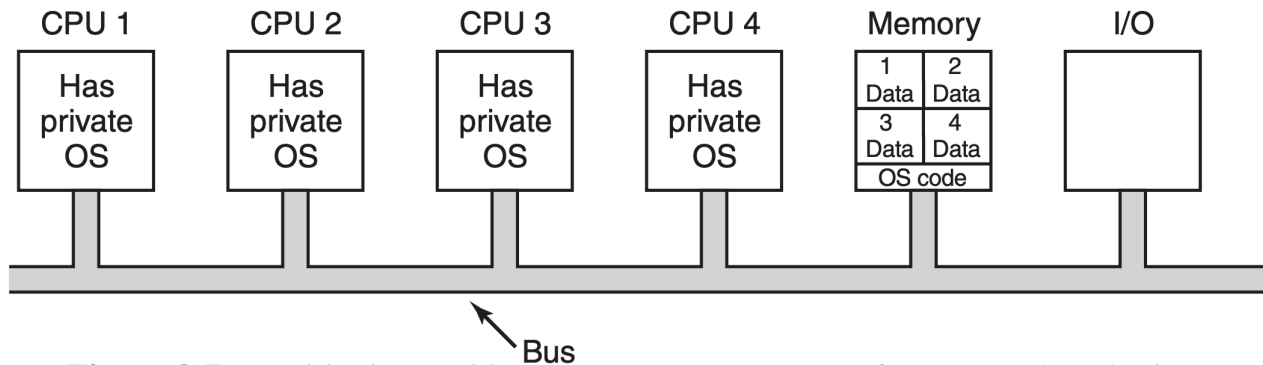


Figure 8-7. Partitioning multiprocessor memory among four CPUs, but sharing a single copy of the operating system code. The boxes marked Data are the operating system's private data for each CPU.

Might still be better than n separate computers

Partitioned memory simplifies coordination across OS instances, but is potentially inefficient...

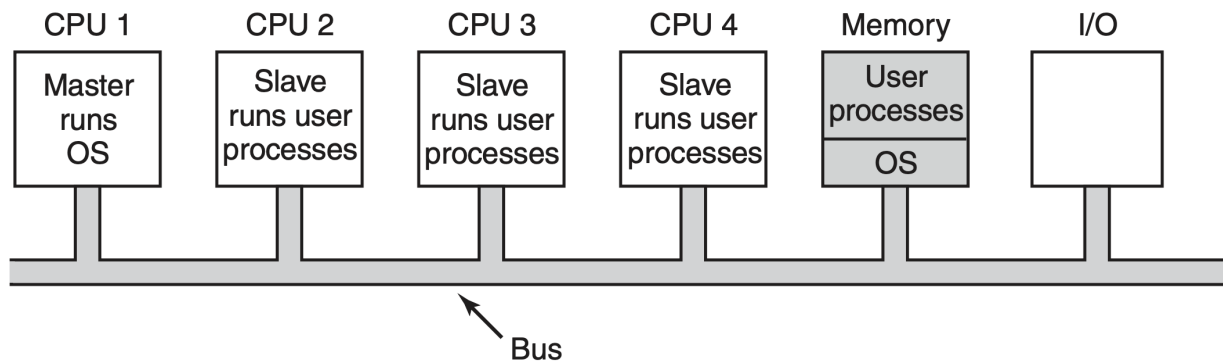
- Load can become imbalanced
- Data can become inconsistent (especially with buffers!)

Multiprocessor hardware – Why is scheduling complicated?

Where to place the OS itself?

All system calls are redirected to the master CPU

Master-Slave



... can easily become a bottleneck ...

Figure 8-8. A master-slave multiprocessor model.

Multiprocessor hardware – Why is scheduling complicated?

Where to place the OS itself?

Symmetric multiprocessors

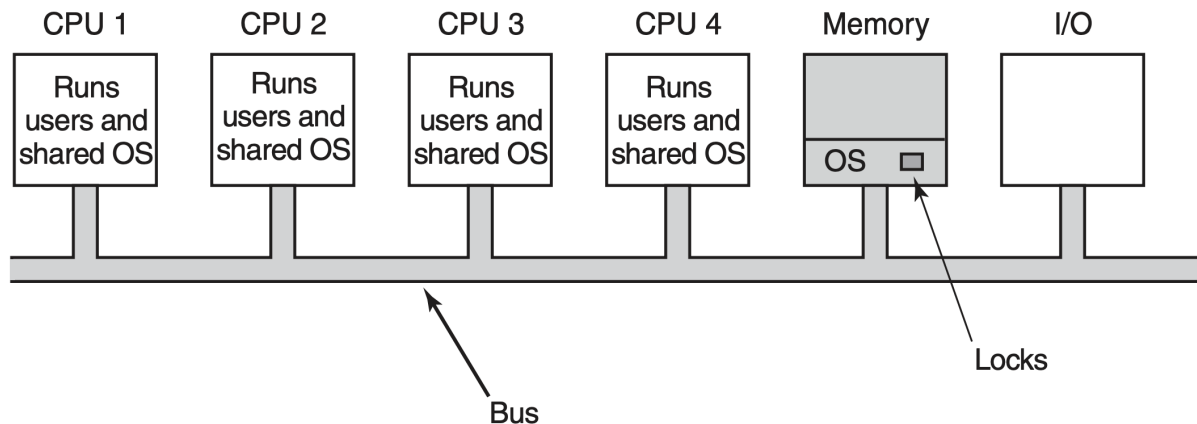


Figure 8-9. The SMP multiprocessor model.

This can balance workload and resources among processes and users

→ But it builds on CONCURRENT ACCESS TO KERNEL!!!

Two threads could modify the same data structure at the same time.

Big lock? → Then the execution of the kernel will be serialized, the kernel itself can become a bottleneck

Critical regions / fine-grained parallelism? → better!
...but makes it hard to program (e.g., deadlocks...)

Agenda

- Introduction
- Scheduling in batch, interactive, and real-time systems
- Process vs. thread scheduling
- Multiprocessor hardware
 - Why does it complicate the matter?
- Multiprocessor scheduling
 - Time sharing
 - Space sharing
 - Gang scheduling

Time sharing – global queue variant

- We could have a single global ready queue

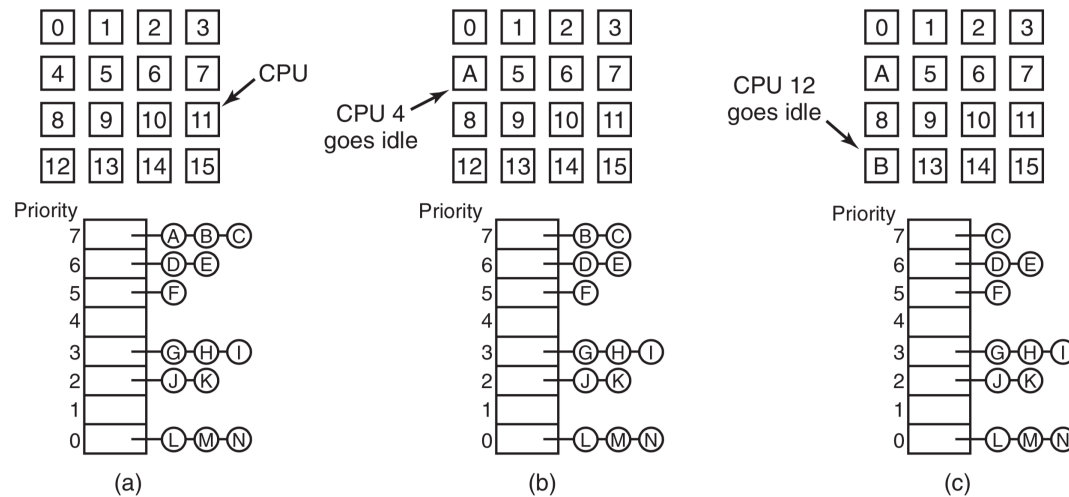


Figure 8-12. Using a single data structure for scheduling a multiprocessor.

- + Automatic load balancing
- Contention might bottleneck the system
- Still suffers from the “affinity problem”

Time sharing – two-level scheduling algorithm

- Each CPU-core has its own collection of threads (assigned at creation time, for example using round-robin or least-loaded)
- Idle CPU-cores can still take threads from other CPU-cores if needed (work stealing)
- Benefits:
 - Load balancing
 - Cache affinity
 - Less contention

Space sharing

- When a set of related threads (e.g., from the same process) is created, the OS tries to schedule all of them at the same time (if enough CPUs are available).
- Under strict space sharing, if a thread issues an I/O request, its allocated processor may remain reserved and idle.

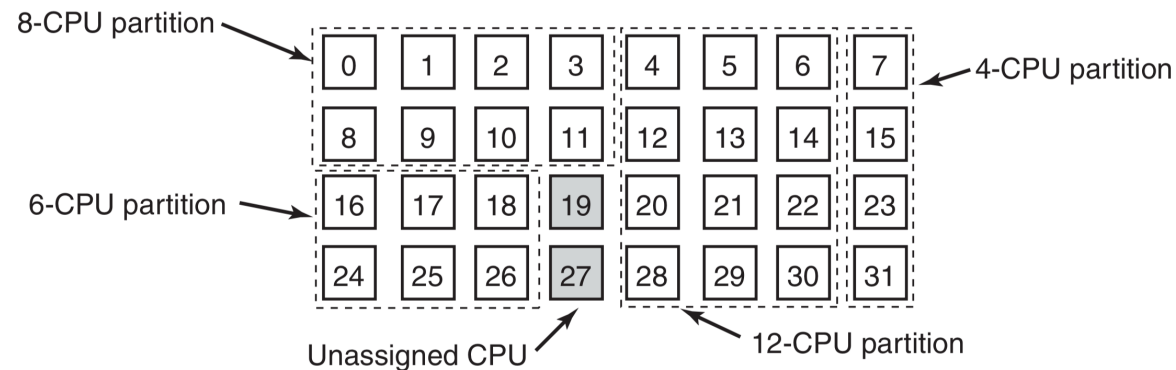


Figure 8-13. A set of 32 CPUs split into four partitions, with two CPUs available.

Gang scheduling

- Schedule both in time and space
- Groups of related threads are scheduled as a gang (with same quantum)
- All gang members run at once
- All gang members start / end their quantum together

Gang scheduling - example

		CPU					
		0	1	2	3	4	5
Time slot	0	A ₀	A ₁	A ₂	A ₃	A ₄	A ₅
	1	B ₀	B ₁	B ₂	C ₀	C ₁	C ₂
	2	D ₀	D ₁	D ₂	D ₃	D ₄	E ₀
	3	E ₁	E ₂	E ₃	E ₄	E ₅	E ₆
	4	A ₀	A ₁	A ₂	A ₃	A ₄	A ₅
	5	B ₀	B ₁	B ₂	C ₀	C ₁	C ₂
	6	D ₀	D ₁	D ₂	D ₃	D ₄	E ₀
	7	E ₁	E ₂	E ₃	E ₄	E ₅	E ₆

Figure 8-15. Gang scheduling.

Questions

31

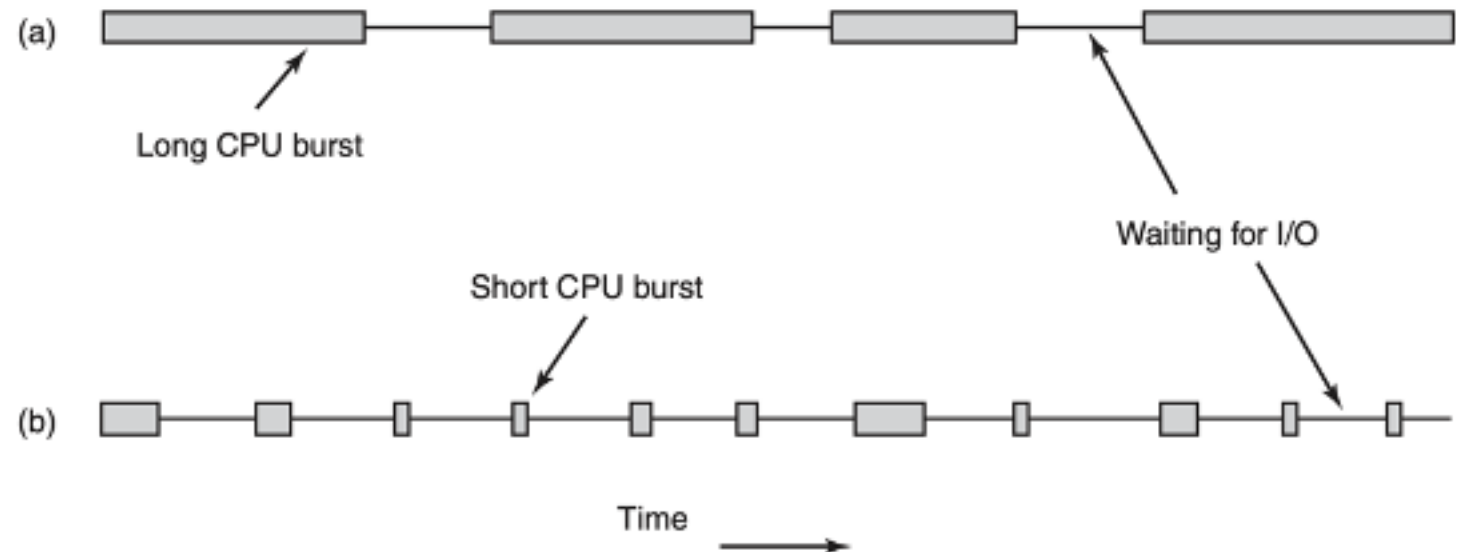
Vincenzo Gulisano

A slow I/O device makes a process accessing such a device I/O bound

1. True
2. False

Which of these 2 processes is CPU-bound and which is I/O-bound?

1. a is CPU, b is I/O
2. b is CPU, a is I/O
3. Is this one of your tricky questions?



The quantum duration in round-robin scheduling indicates how long the process will run

1. Yes
2. No

Kernel threads are expected to perform better than user threads when a multithreaded process has one thread that is heavily I/O-bound

1. Yes
2. No

Kernel threads are expected to perform better than user threads when a multithreaded process has one thread that is heavily I/O-bound

1. I said yes before but now I have doubts
2. I still believe the answer is yes

Setup: CPU-bound processes that use 100% of their quantum, I/O-bound processes, priority + round-robin scheduling. To maximize responsiveness:

1. higher priority for CPU-bound procs
2. higher priority for I/O-bound procs